

Week 3: MC, TD, and the Road to Deep RL

FYS4411 – Reinforcement Learning and Agentic AI

University of Oslo, Department of Physics

June 2026

What this lecture is trying to do

- ▶ Week 1 introduced the MDP language: states, actions, rewards, policies, returns, and values.
- ▶ Week 2 connected Bellman equations to dynamic programming and then to Monte Carlo learning.
- ▶ Week 3 completes the basic model-free tabular story and uses it to motivate deep RL.

Main question

How can an agent learn useful values and policies from sampled experience when the model $p(s', r \mid s, a)$ is unavailable?

1. Monte Carlo as complete-return learning.
2. Temporal-difference learning as one-step bootstrapping.
3. SARSA and Q-learning as on-policy and off-policy TD control.
4. Applications: quantum control, molecular dynamics, and robotics/control.
5. Why tabular methods break and why neural networks enter.
6. Overview of DQN, policy gradients, and actor-critic methods.

The recurring update template

A large part of model-free RL can be organized by the same update pattern:

$$\text{new estimate} = \text{old estimate} + \alpha [\text{target} - \text{old estimate}].$$

The difference between algorithms is mostly the choice of target:

Monte Carlo	:	G_t
TD prediction	:	$R_{t+1} + \gamma V(S_{t+1})$
SARSA	:	$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$
Q-learning	:	$R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a').$

From model-based to sampled experience

In dynamic programming, the environment model is available and we can compute expectations:

$$v_{\pi}(s) = \sum_{a \in A(s)} \pi(a | s) \sum_{s' \in S} \sum_{r \in R} p(s', r | s, a) [r + \gamma v_{\pi}(s')].$$

In model-free RL, the kernel $p(s', r | s, a)$ is not directly available. The agent instead observes samples

$$(S_t, A_t, R_{t+1}, S_{t+1}).$$

Takeaway

The mathematical goal remains expected return; the computational problem changes from exact expectations to estimates from data.

Notation recap: the trajectory

A generated episode has the form

$$S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T, S_T.$$

For policy π and environment kernel p , a finite trajectory has probability

$$P_\pi(\tau) = \mu_0(s_0) \prod_{t=0}^{T-1} \pi(a_t \mid s_t) p(s_{t+1}, r_{t+1} \mid s_t, a_t).$$

Changing the policy changes the distribution over future data.

Return and delayed consequences

The return from time t is

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

For continuing tasks one writes

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots.$$

The key recursion is

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

This recursion is the source of Bellman equations and TD targets.

Prediction and control

Prediction

Given a fixed policy π , estimate

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

or

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

Question: how good is this policy?

Control

Find or improve a policy. A greedy improvement from action values is

$$\pi'(s) \in \arg \max_{a \in A(s)} Q(s, a).$$

Question: what should the agent do?

Monte Carlo learning: core idea

Monte Carlo methods estimate value functions from complete sampled episodes.

Prediction target

If state s is visited at time t , then the observed return G_t is one sample of the random variable whose expectation is

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

If we observe returns $G^{(1)}(s), \dots, G^{(N(s))}(s)$, then

$$V(s) = \frac{1}{N(s)} \sum_{i=1}^{N(s)} G^{(i)}(s).$$

Why averaging returns is justified

For a fixed policy π , each return following a visit to s is a sample from the conditional return distribution.

The target value is an expectation:

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

The Monte Carlo estimate is the empirical mean:

$$V_N(s) = \frac{1}{N} \sum_{i=1}^N G^{(i)}(s).$$

Takeaway

The law of large numbers is the basic statistical justification: repeated sampled returns approximate the expected return.

Incremental Monte Carlo update

Storing all returns is unnecessary. If $V_{n-1}(s)$ is the average after $n - 1$ returns and $G_n(s)$ is a new return, then

$$V_n(s) = V_{n-1}(s) + \frac{1}{n} [G_n(s) - V_{n-1}(s)].$$

For action values:

$$Q_n(s, a) = Q_{n-1}(s, a) + \frac{1}{n} [G_n(s, a) - Q_{n-1}(s, a)].$$

Often one uses a step size α :

$$Q(s, a) \leftarrow Q(s, a) + \alpha [G - Q(s, a)].$$

First-visit and every-visit Monte Carlo

A state may appear more than once in one episode.

First-visit MC

Use G_t only for the first time $S_t = s$ occurs in an episode:

$$S_j \neq s \quad \text{for all } 0 \leq j < t.$$

Each state contributes at most one return per episode.

Every-visit MC

Use G_t every time $S_t = s$ occurs. The same episode may contribute multiple returns for the same state.

Both estimate the same value function under suitable sampling.

Monte Carlo control

For control, action values are more directly useful than state values:

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

The control loop is:

1. Generate an episode using the current policy.
2. Compute returns after the episode ends.
3. Update $Q(s, a)$ toward the observed returns.
4. Improve the policy using the updated action values.

A greedy improvement would be

$$\pi'(s) \in \arg \max_{a \in A(s)} Q(s, a).$$

The exploration problem

If the agent always chooses the current greedy action, it may never learn that other actions are better.

ϵ -greedy policy

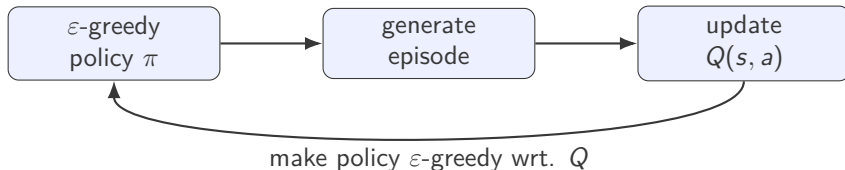
For a unique greedy action $a^* \in \arg \max_b Q(s, b)$,

$$\pi(a \mid s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A(s)|}, & a = a^*, \\ \frac{\epsilon}{|A(s)|}, & a \neq a^*. \end{cases}$$

This preserves exploration while concentrating probability on the best-known action.

On-policy Monte Carlo control

In on-policy learning, the same policy generates the episodes and is improved from the observed returns.



Takeaway

Monte Carlo control is natural when rewards are terminal or delayed until an episode ends.

Off-policy Monte Carlo: the idea

Sometimes the data are generated by one policy, but we want to evaluate another.

Behavior policy	Target policy
$b(a s)$ generates episodes. It is often more exploratory.	$\pi(a s)$ is the policy whose value we want. It may be greedy or less exploratory.

The importance-sampling ratio over a trajectory segment is

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}.$$

This correction can have high variance.

Blackjack: a clean Monte Carlo example

Blackjack is naturally episodic: a game ends, and the final result determines the return.

- ▶ State: $s = (\text{player sum}, \text{dealer card}, \text{usable ace})$.
- ▶ Actions: $A = \{\text{stick}, \text{hit}\}$, often encoded as 0, 1.
- ▶ Rewards: win = +1, draw = 0, loss = -1.
- ▶ Discount: $\gamma = 1$.

Since intermediate rewards are zero, the terminal reward is the return for each visited state-action pair.

Blackjack and action values

The control objective is to estimate

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

from sampled games.

The tabular representation in the notes uses the shape

$$(22, 11, 2, 2),$$

corresponding to player sum, dealer card, usable ace, and action.

What this example teaches

The algorithm does not need the card-game transition probabilities. It needs only complete sampled episodes and returns.

Monte Carlo in quantum control

A finite pulse sequence is naturally episodic. The agent applies controls and observes final performance.

$$|\psi_{t+1}\rangle = e^{-iH(A_t)\Delta t}|\psi_t\rangle.$$

A terminal fidelity reward is

$$R_T = F_T = |\langle\psi_{\text{target}}|\psi_T\rangle|^2.$$

- ▶ State: discretized Bloch vector, wavefunction features, or action history.
- ▶ Action: pulse amplitude, phase, duration, or discrete bang-bang pulse.
- ▶ Reward: terminal fidelity, possibly with pulse penalties.

Reference: See application discussion in Week 3 notebook; quantum-control motivation also aligns with [arXiv:2601.18953](https://arxiv.org/abs/2601.18953).

Monte Carlo in molecular dynamics

For MD, Monte Carlo RL is useful when the objective is attached to a complete trajectory.

Examples of episodic objectives:

- ▶ end in a low-energy configuration;
- ▶ reach a target conformation;
- ▶ complete a rare transition;
- ▶ follow a switching protocol with low work.

A terminal reward might be

$$R_T = -u_T - \lambda \sum_{t=0}^{T-1} \text{control cost}_t,$$

where u_T is the final potential energy per particle.

Important: the RL action must be an added control or decision variable; the Lennard-Jones force law itself is not replaced by RL.

Limitations of Monte Carlo methods

Monte Carlo methods are simple and model-free, but they have important limitations.

- ▶ Updates usually wait until the episode terminates.
- ▶ Complete returns can have high variance.
- ▶ Long episodes delay credit assignment.
- ▶ Rare successful events produce sparse learning signals.
- ▶ Continuing tasks require additional handling.

Takeaway

This motivates temporal-difference learning: update from each transition instead of waiting for the entire return.

Discussion: terminal rewards

Discussion

In a quantum-control experiment, suppose the only reliable measurement is the final target-state fidelity. Is Monte Carlo learning a natural choice? What information is lost compared with observing intermediate fidelities?

Discussion

In molecular dynamics, what would count as an “episode”? A fixed simulation length? A rare-event crossing? A cooling schedule?

Temporal-difference learning: motivation

Monte Carlo uses the complete return

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Temporal-difference learning replaces this by a one-step target:

$$R_{t+1} + \gamma V(S_{t+1}).$$

Why this matters

The agent can update immediately after observing one transition

$$(S_t, A_t, R_{t+1}, S_{t+1}),$$

without waiting for the episode to end.

TD from the Bellman equation

The Bellman expectation equation says

$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s].$$

TD learning replaces the exact expectation by the sampled transition:

$$\text{target} = R_{t+1} + \gamma V(S_{t+1}).$$

The TD(0) update is

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)].$$

If S_{t+1} is terminal, the continuation value is set to zero.

The temporal-difference error

The TD error is

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t).$$

If $\delta_t > 0$

The observed transition was better than expected.
Increase $V(S_t)$.

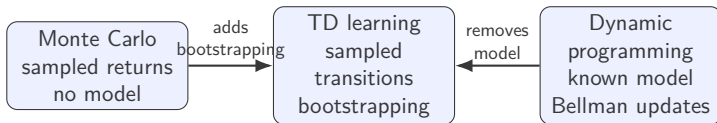
If $\delta_t < 0$

The observed transition was worse than expected.
Decrease $V(S_t)$.

The update is simply

$$V(S_t) \leftarrow V(S_t) + \alpha \delta_t.$$

TD as a bridge between MC and DP



- ▶ Like MC: learns from sampled experience.
- ▶ Like DP: uses a Bellman-style one-step update.
- ▶ Unlike MC: does not need complete returns.
- ▶ Unlike DP: does not need $p(s', r \mid s, a)$.

MC target versus TD target

Method	Target	Consequence
Monte Carlo	G_t	Complete return, no bootstrapping
TD(0)	$R_{t+1} + \gamma V(S_{t+1})$	One-step update, bootstrapping
SARSA	$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$	On-policy TD control
Q-learning	$R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$	Off-policy TD control

The target choice determines what is being learned and how quickly information propagates.

Bias and variance intuition

Monte Carlo targets use actual complete returns. They are often high variance but do not bootstrap from current value estimates.

TD targets use one observed reward and one estimated value. They may have lower variance, but they introduce bias from the current estimate.

$$\begin{aligned}\text{MC target} &= G_t \\ \text{TD target} &= R_{t+1} + \gamma V(S_{t+1}).\end{aligned}$$

Takeaway

TD trades complete-return information for earlier, more frequent, bootstrapped updates.

From prediction to control

TD prediction estimates values for a fixed policy. TD control learns action values and improves the policy.

Action-value function

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

The agent must learn from transitions that include actions:

$$S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}.$$

This leads directly to SARSA.

SARSA: on-policy TD control

SARSA learns action values using the next action actually selected by the current policy.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)] .$$

Why the name SARSA?

The update uses the quintuple

$$S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1} .$$

Because A_{t+1} comes from the current behavior policy, SARSA is on-policy.

SARSA and exploratory behavior

If the behavior policy is ε -greedy, SARSA learns values for that exploratory policy.

This matters when exploration is risky.

- ▶ The next action A_{t+1} may be non-greedy.
- ▶ The target includes that actual next action.
- ▶ Therefore the learned value reflects the consequences of continued exploration.

Takeaway

SARSA answers: how good is this action if I keep behaving according to my current exploratory policy?

Q-learning: off-policy TD control

Q-learning updates toward the greedy next action, regardless of how the data were generated:

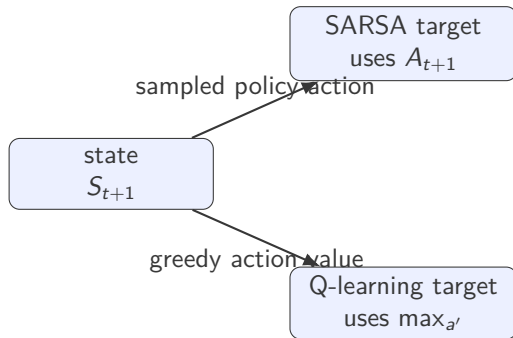
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right].$$

The behavior policy may still be ε -greedy, but the target policy is greedy.

Takeaway

Q-learning answers: how good is this action if I behave greedily from the next state onward?

On-policy versus off-policy in one picture



- ▶ SARSA: target follows the behavior policy.
- ▶ Q-learning: target follows the greedy policy.
- ▶ Both are model-free TD control methods.

Expected SARSA

Expected SARSA replaces the sampled next action by an expectation under the policy:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_{a'} \pi(a' | S_{t+1}) Q(S_{t+1}, a') - Q(S_t, A_t) \right].$$

- ▶ SARSA samples A_{t+1} .
- ▶ Expected SARSA averages over possible A_{t+1} .
- ▶ Q-learning replaces the average by a maximum.

This helps organize several TD methods as different choices of next-state target.

CliffWalking: why SARSA and Q-learning differ

In CliffWalking, the shortest route runs along a dangerous cliff.

SARSA

Because it evaluates the exploratory policy, it accounts for random exploratory actions near the cliff. It often learns a safer route.

Q-learning

Because it targets the greedy continuation, it often learns the shortest greedy route near the cliff. This route can be risky during exploration.

Reference: This is the qualitative interpretation presented in the RL notes' CliffWalking discussion.

Physical interpretation of on-policy versus off-policy

In physical control, the distinction has a concrete meaning.

- ▶ On-policy methods value the controller as it is actually deployed during learning.
- ▶ Off-policy methods may learn about a cleaner or greedier target controller while using exploratory data.

Example

A robot trained with random exploratory torques may fall near unsafe states. SARSA includes this risk in the target; Q-learning evaluates the greedy target policy that would not intentionally take the exploratory torque.

Application map: three pillars

Domain	State	Action	Reward
Quantum control	Bloch vector, fidelity, pulse index	discrete or continuous pulse	final or incremental fidelity
Molecular dynamics	energy, temperature, collective variables	steering force, schedule, bias	low energy, transition success
Robotics/control	position, velocity, angle	force, torque, acceleration	target reached, stable motion

The formalism is shared; the physics determines what S_t , A_t , and R_{t+1} mean.

Week 3 quantum example: robust qubit inversion

The notebook formulates a two-level control problem:

$$|\psi_0\rangle = |0\rangle, \quad |\psi_{\text{target}}\rangle = |1\rangle.$$

The Hamiltonian is

$$H_t = \frac{\Delta_t}{2}\sigma_z + \frac{u_x(A_t)}{2}\sigma_x + \frac{u_y(A_t)}{2}\sigma_y.$$

The state evolves by

$$|\psi_{t+1}\rangle = e^{-iH_t\Delta t}|\psi_t\rangle.$$

Detuning noise makes the same pulse sequence produce variable outcomes.

Qubit actions and tabular state

The action set is discrete:

$$A = \{(0, 0), (\Omega, 0), (-\Omega, 0), (0, \Omega), (0, -\Omega)\}.$$

The continuous quantum state is converted to a tabular observation using Bloch components:

$$x_t = \langle \sigma_x \rangle, \quad y_t = \langle \sigma_y \rangle, \quad z_t = \langle \sigma_z \rangle.$$

Then

$$S_t = (t, \text{bin}(x_t), \text{bin}(y_t), \text{bin}(z_t)).$$

Including t makes the finite-horizon task Markov at the tabular level.

Qubit rewards: terminal and shaped

The target fidelity is

$$F_t = |\langle 1 | \psi_t \rangle|^2.$$

Monte Carlo can use the terminal reward

$$R_N = F_N - \lambda \sum_{t=0}^{N-1} \|u(A_t)\|^2 \Delta t.$$

TD methods can use shaped rewards

$$R_{t+1} = F_{t+1} - F_t - \lambda \|u(A_t)\|^2 \Delta t.$$

For $\gamma = 1$, the fidelity increments telescope:

$$\sum_{t=0}^{N-1} (F_{t+1} - F_t) = F_N - F_0.$$

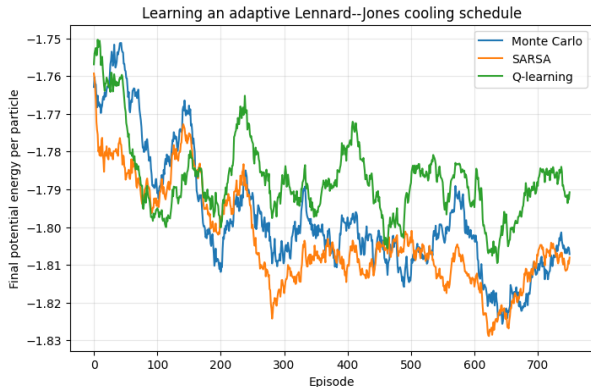
Qubit example: what the notebook demonstrates

- ▶ Monte Carlo uses final fidelity and updates after the full pulse sequence.
- ▶ SARSA and Q-learning use fidelity increments and update after each pulse.
- ▶ The state is coarse and tabular; it is not the full quantum state.
- ▶ Noise creates a genuine learning problem because repeated trials differ.

Bridge to deep RL

If the observation were the full wavefunction or density-matrix features, tabular methods would quickly become impractical.

Qubit example: learned performance



Notebook output from the Week 3 quantum-control experiment. The precise values depend on random seeds and discretization, but the didactic point is the comparison of complete-return and TD-style learning.

Week 3 MD example: Lennard-Jones dynamics

The MD part begins with the Lennard-Jones potential

$$U(r) = 4\varepsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right].$$

The total potential energy is

$$U_{\text{tot}} = \sum_{i < j} U(r_{ij}).$$

The RL agent does **not** replace the force law or integrator. It controls an added decision variable in a simulation workflow.

Reference: The notebook labels this explicitly as an RL extension of a Lennard-Jones MD system.

MD example: what is the RL task?

The notebook introduces an adaptive annealing/control problem.

- ▶ State: coarse bins of temperature, potential energy per particle, time, and previous action.
- ▶ Action: select a target temperature for a weak rescaling step.
- ▶ Environment: Lennard-Jones MD evolution over several integration steps.
- ▶ Objective: end with low potential energy while avoiding excessive intervention.

A terminal objective has the form

$$R_H = -u_H - \lambda \sum_{t=0}^{H-1} (T^{(A_t)} - T_t)^2.$$

This is a pedagogical RL extension, not a claim that thermostat control is part of the original LJ model.

MD reward shaping and TD learning

A shaped reward can be

$$R_{t+1} = u_t - u_{t+1} - \lambda(T^{(A_t)} - T_t)^2.$$

The energy term telescopes:

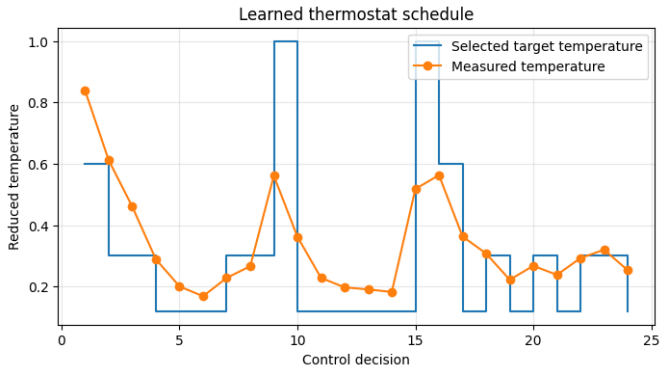
$$\sum_{t=0}^{H-1} (u_t - u_{t+1}) = u_0 - u_H.$$

If the initial distribution is fixed, maximizing shaped return is aligned with lowering final energy plus control cost.

Why TD fits this case

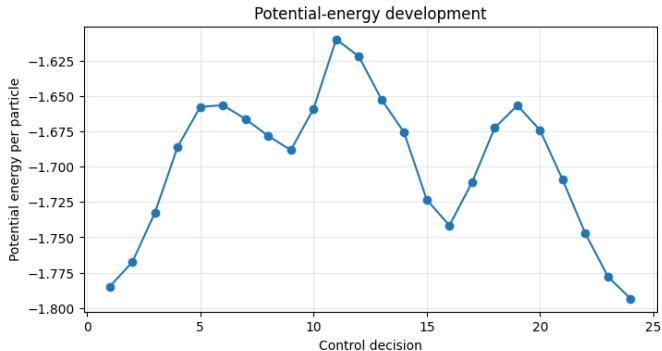
TD methods use the intermediate energy changes, rather than waiting for the final configuration only.

MD example: learning curves



Notebook output comparing final potential energy trajectories for Monte Carlo, SARSA, and Q-learning in the pedagogical LJ annealing extension.

MD example: learned schedule



Notebook output showing the selected target temperature and measured temperature during one greedy rollout.

Control theory / robotics: why TD is natural

In robotics and classical control, episodes can be long and actions affect the next state immediately.

- ▶ State: position, velocity, angle, angular velocity.
- ▶ Action: force, torque, acceleration, steering.
- ▶ Reward: progress, stability, energy cost, safety.

A one-dimensional navigation task can use

$$S_t = (\text{bin}(x_t), \text{bin}(v_t)), \quad A_t \in \{-F, 0, +F\}.$$

TD methods can update after each movement step, making them more suitable than pure terminal-return MC for real-time control.

Interactive checkpoint: choose the algorithm

Discussion

A qubit pulse experiment only gives a final success/failure measurement. Which method is most natural first: Monte Carlo, TD prediction, SARSA, or Q-learning?

Discussion

A robot receives a reward after every movement step and must avoid unsafe exploratory actions. Would SARSA or Q-learning better reflect the behavior policy during training?

Discussion

An MD simulation produces an energy change after every control action. What reward shaping could make TD useful?

Why tabular methods eventually fail

Tabular methods store one entry per state or state-action pair:

$$V(s), \quad Q(s, a).$$

This works for small finite problems, but not for:

- ▶ continuous positions and velocities;
- ▶ high-dimensional molecular configurations;
- ▶ quantum wavefunction or density-matrix features;
- ▶ robot sensors and images;
- ▶ large action histories or partial observations.

Takeaway

We need a representation that generalizes across similar states.

Function approximation

Replace tables by parametrized functions.

$$\begin{aligned} V(s) &\longrightarrow \hat{v}(s; w), \\ Q(s, a) &\longrightarrow \hat{q}(s, a; w), \\ \pi(a | s) &\longrightarrow \pi_{\theta}(a | s). \end{aligned}$$

Here w and θ are trainable parameters. A neural network is one flexible way to represent these functions.

Deep RL in one sentence

Deep RL is tabular RL with value functions or policies replaced by neural networks and trained by gradient-based optimization.

What changes and what does not

Still the same RL objects

- ▶ trajectories
- ▶ returns G_t
- ▶ policies π
- ▶ value functions v_π, q_π
- ▶ Bellman targets
- ▶ TD errors

New representation

- ▶ tables become functions;
- ▶ updates become gradient steps;
- ▶ one sample can affect many states;
- ▶ optimization stability becomes central.

Supervised-learning-like losses in RL

With function approximation, a TD target becomes a regression target.
For Q-learning, define

$$Y_t = R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a'; w).$$

Then use the squared loss

$$L_t(w) = [Y_t - \hat{q}(S_t, A_t; w)]^2.$$

The gradient step moves the network prediction toward the target:

$$w \leftarrow w - \alpha \nabla_w L_t(w).$$

But unlike supervised learning, the target is generated by interaction and may depend on current estimates.

Deep Q-learning replaces the Q-table by a neural network.

For a finite action set $A = \{a_1, \dots, a_m\}$, the network can output all action values:

$$\hat{q}(s, a_1; w), \dots, \hat{q}(s, a_m; w).$$

A greedy policy is

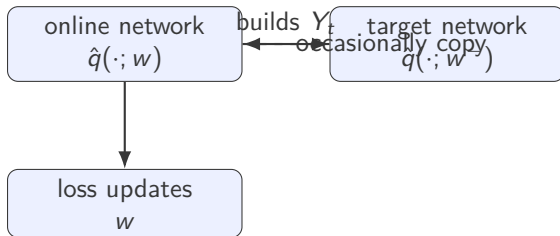
$$\pi(s) \in \arg \max_{a \in A(s)} \hat{q}(s, a; w).$$

The DQN target commonly uses a separate target network:

$$Y_t = R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a'; w^-).$$

Why a target network?

If the same network creates the target and is trained toward the target, the target moves rapidly.



A slowly updated w^- stabilizes the target.

Experience replay

Consecutive transitions in one trajectory are highly correlated. Experience replay stores transitions in a buffer:

$$\mathcal{D} = \{(S_t, A_t, R_{t+1}, S_{t+1})\}.$$

Training samples minibatches from $\mathcal{D}$.

- ▶ Reduces temporal correlations in updates.
- ▶ Reuses previous experience.
- ▶ Makes the optimization closer to ordinary minibatch training.

Takeaway

Target networks and replay buffers are not new RL objectives; they are stability devices for neural-network value learning.

Policy parametrization

Value-based methods learn a value function and derive a policy.

Policy-based methods represent the policy directly.

For finite actions, a network outputs logits $z_\theta(s, a)$ and a softmax policy is

$$\pi_\theta(a \mid s) = \frac{\exp(z_\theta(s, a))}{\sum_{b \in A(s)} \exp(z_\theta(s, b))}.$$

The objective becomes

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_0], \quad \theta^* \in \arg \max_{\theta} J(\theta).$$

Changing θ changes the trajectory distribution itself.

Why learn the policy directly?

Direct policy parametrization is useful because:

- ▶ stochastic policies are natural for exploration;
- ▶ continuous actions can be represented without an explicit $\arg \max_a Q(s, a)$;
- ▶ the policy can be trained for the exact performance objective;
- ▶ actor-critic methods can combine policy and value learning.

For physical control, the action may be a pulse amplitude, force, torque, or steering signal.

Policy gradient objective

For a finite trajectory

$$\tau = (s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T, s_T),$$

write

$$J(\theta) = \sum_{\tau} P_{\pi_{\theta}}(\tau) G_0(\tau).$$

Taking the gradient gives

$$\nabla_{\theta} J(\theta) = \sum_{\tau} \nabla_{\theta} P_{\pi_{\theta}}(\tau) G_0(\tau).$$

Use the log-derivative identity:

$$\nabla_{\theta} P_{\pi_{\theta}}(\tau) = P_{\pi_{\theta}}(\tau) \nabla_{\theta} \log P_{\pi_{\theta}}(\tau).$$

The key policy-gradient simplification

The trajectory probability contains policy factors and environment factors:

$$P_{\pi_{\theta}}(\tau) = \mu_0(s_0) \prod_{t=0}^{T-1} \pi_{\theta}(a_t \mid s_t) p(s_{t+1}, r_{t+1} \mid s_t, a_t).$$

Only the policy depends on θ . Therefore

$$\nabla_{\theta} \log P_{\pi_{\theta}}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t \mid S_t).$$

Takeaway

Policy gradients can be model-free: we do not differentiate the environment dynamics.

The policy-gradient theorem in sampled form gives the update

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t.$$

Interpretation:

- ▶ if an action is followed by high return, increase its log probability;
- ▶ if an action is followed by low return, decrease its log probability;
- ▶ the environment is treated as a source of sampled trajectories.

REINFORCE is simple, but complete-return gradients often have high variance.

Baselines and advantages

A baseline can reduce variance without changing the expected policy gradient:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t)(G_t - b(S_t))].$$

A natural baseline is $b(S_t) = v_{\pi}(S_t)$.

The advantage function is

$$A_{\pi}(s, a) = q_{\pi}(s, a) - v_{\pi}(s).$$

It asks: how much better was this action than the typical action from the same state?

Actor-critic methods

Actor-critic methods learn both a policy and a value function.

Actor	Critic
The actor is the policy $\pi_{\theta}(a \mid s)$.	The critic is a value estimate such as $\hat{v}(s; w)$.
It chooses actions.	It evaluates the actor.
The critic supplies a lower-variance learning signal for the actor.	

TD error as actor signal

A one-step critic error is

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; w) - \hat{v}(S_t; w).$$

This TD error can act as an advantage estimate.

Actor update:

$$\theta \leftarrow \theta + \alpha_\theta \delta_t \nabla_\theta \log \pi_\theta(A_t | S_t).$$

Critic update:

$$w \leftarrow w + \alpha_w \delta_t \nabla_w \hat{v}(S_t; w).$$

Takeaway

The actor learns what to do; the critic learns how good the actor's states or actions are.

Deterministic policy gradients: preview

For continuous control, a deterministic actor outputs one action directly:

$$a = \mu_{\theta}(s).$$

A critic estimates $Q^{\mu}(s, a)$. The deterministic policy gradient has the chain-rule form

$$\nabla_{\theta} J(\theta) = \mathbb{E}_s \left[\nabla_{\theta} \mu_{\theta}(s) \nabla_a Q^{\mu}(s, a) \Big|_{a=\mu_{\theta}(s)} \right].$$

This is relevant for continuous physical controls: torques, voltages, pulse amplitudes, steering, and biasing forces.

Deep RL and the three pillars

Domain	Why tables break	Natural deep-RL representation
Quantum tech	wavefunctions, density matrices, pulse histories	neural policy over pulse amplitudes or phases
Molecular dynamics	many coordinates, velocities, collective variables	neural value/policy on coarse or learned features
Robotics/control	continuous states and actions	actor-critic or deterministic policy gradients

Deep RL is motivated by scale and generalization, not by a change in the RL objective.

Practical issues in deep RL

Neural-network RL is powerful, but less stable than tabular RL.

- ▶ Targets may move during training.
- ▶ Consecutive samples are correlated.
- ▶ The data distribution changes as the policy changes.
- ▶ Bootstrapped targets can propagate errors.
- ▶ Exploration remains difficult.
- ▶ Reward scale and shaping matter strongly.

Takeaway

Much of modern deep RL is about stabilizing the same basic MC/TD ideas.

References for applications

- ▶ Course notes: *Reinforcement Learning and Agentic AI*, Ch. 2 and Ch. 3.1–3.6.
- ▶ Week 3 notebook: qubit inversion and Lennard-Jones annealing examples.
- ▶ Quantum-control RL review: [arXiv:2601.18953](https://arxiv.org/abs/2601.18953).
- ▶ Agentic AI and quantum control notes: RL controller + simulator + agentic workflow architecture.

When adding external papers to a lecture, separate three things clearly:

1. what is in the paper;
2. what is in the course notes;
3. what is a pedagogical simplification used in the notebook.

What students should take away

1. Monte Carlo learns from complete returns.
2. TD learns from one-step bootstrapped targets.
3. SARSA is on-policy; Q-learning is off-policy.
4. Reward design changes the learning signal, especially in physical systems.
5. Function approximation replaces tables by trainable functions.
6. Deep RL keeps the same values, policies, returns, and Bellman targets, but adds neural-network optimization and stability issues.

Discussion

Suppose you are designing an RL controller for a quantum experiment, an MD simulation, or a robot. What information can be observed after each action? What can only be observed at the end?

Discussion

Is the state representation Markov? If not, what information is missing?

Discussion

Would you prefer Monte Carlo, SARSA, Q-learning, DQN, or an actor-critic method as a first attempt? Why?